

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Varsha Vaidhina S	Reg.No.:	192511488
Department:	B.E - CSE	Date:	

80% 

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1	3	3	4	4	2 16
Q2	4	4	4	4	2 18
Q3	3	3	3	3	3 15
Q4	4	4	4	4	2 18
Q5	3	3	3	3	1 13

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately(4)	Understands problem with minor gaps(3)	Partial understanding; misses key elements(2)	Misinterprets or unable to understand problem(1)
Method Selection	20%	Selects most appropriate method/formula with strong justification(4)	Selects correct method with minor errors(3)	Inappropriate or partially correct method(2)	Unable to select method(1)
Solution Process	25%	Logical, systematic steps with correct approach throughout(5)	Mostly correct steps with minor mistakes(4)	Disorganized steps; multiple errors(3)	No clear process(0)
Accuracy of Calculation	20%	All calculations accurate; correct final answer(4)	Minor calculation errors(3)	Frequent errors; incorrect final answer(2)	No valid calculations(0)
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance(3)	Basic interpretation with minor omissions(2)	Limited or unclear interpretation(1)	No interpretation(0)

80

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	S.VARSHA VARDHINI	Reg.No.:	192511488
Department:	B.E.CSE	Date:	12-08-2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

SIGNED 8-BIT ARITHMETIC USING 2'S COMPLEMENT

Given two signed 8-bit numbers:

First number = +45

Second number = -23

We have to perform:

1. $(+45) + (-23)$

2. $(+45) - (-23)$

STEP 1: REPRESENT +45 IN 8-BIT BINARY

$$45 = 32 + 8 + 4 + 1$$

Therefore,

$$+45 = 00101101$$

STEP 2: REPRESENT -23 IN 8-BIT 2'S COMPLEMENT

First, convert +23 into binary:

$$23 = 16 + 4 + 2 + 1$$

$$+23 = 00010111$$

Take the 1's complement:

$$00010111$$

$$11101000$$

Add 1 to obtain the 2's complement:

$$11101000$$

$$+ \quad \quad 1$$

$$11101001$$

Therefore,

$$-23 = 11101001$$

1. **ADDITION:** $(+45) + (-23)$

Write the two numbers:

$$00101101$$

$$+ 11101001$$

$$1\ 00010110$$

The carry generated beyond the 8th bit is discarded.

Therefore,

$$00010110$$

Convert the result into decimal:

$$00010110 = 16 + 4 + 2$$

$$= 22$$

Therefore,

$$(+45) + (-23) = +22$$

OVERFLOW CHECK:

The two operands have different signs:

+45 is positive.

-23 is negative.

In signed 2's complement addition, overflow cannot occur when the two operands have different signs.

Therefore,

Overflow = No

2. SUBTRACTION: $(+45) - (-23)$

In 2's complement arithmetic, subtraction is performed by adding the 2's complement of the number being subtracted.

Therefore,

$$(+45) - (-23)$$

$$= (+45) + (+23)$$

We know:

$$+45 = 00101101$$

$$+23 = 00010111$$

Perform binary addition:

$$\begin{array}{r} 00101101 \\ + 00010111 \\ \hline \end{array}$$

$$01000100$$

Therefore,

$$\text{Result} = 01000100$$

Convert the result into decimal:

$$01000100 = 64 + 4$$

$$= 68$$

Therefore,

$$(+45) - (-23) = +68$$

OVERFLOW CHECK:

The operation is effectively:

$$+45 + +23 = +68$$

Both operands are positive.

The result is also positive:

$$01000100$$

The sign bit is 0, which represents a positive number.

The range of signed 8-bit 2's complement numbers is:

$$-128 \text{ to } +127$$

Since +68 is within this range, no overflow occurs.

Therefore,

Overflow = No

HOW THE PROCESSOR DETECTS OVERFLOW:

In signed 2's complement arithmetic, overflow occurs when two numbers with the same sign produce a result with the opposite sign.

For addition:

1. Positive + Positive = Negative → Overflow

2. Negative + Negative = Positive → Overflow

3. Positive + Negative → No overflow

Overflow can also be detected using the carry into and carry out of the sign bit:

Overflow = Carry into MSB XOR Carry out of MSB

$$V = C_7 \text{ XOR } C_8$$

FINAL ANSWERS:

1. $(+45) + (-23) = +22$

Binary result = 00010110

Overflow = No

2. $(+45) - (-23) = +68$

Binary result = 01000100

Overflow = No

CONCLUSION:

The processor uses 8-bit 2's complement representation to perform signed arithmetic operations efficiently. For the given values, +45 is represented as 00101101 and -23 as 11101001. The addition produces 00010110, which represents +22, while the subtraction produces 01000100, which represents +68. Both results are within the valid signed 8-bit range of -128 to +127. Therefore, no overflow occurs in either operation. The processor detects overflow by examining the sign bits and the carry into and out of the most significant bit. Thus, the final results are correctly interpreted as +22 and +68.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

4-BIT CARRY LOOK-AHEAD ADDER (CLA)

Given two 4-bit binary numbers:

A = 1011

B = 0110

We have to perform the addition using a 4-bit Carry Look-Ahead Adder (CLA).

STEP 1: CALCULATE GENERATE AND PROPAGATE

For each bit:

Generate:

$G_i = A_i \text{ AND } B_i$

Propagate:

$P_i = A_i \text{ XOR } B_i$

The numbers are:

$$A = 1011$$

$$B = 0110$$

From right to left:

$$A_0 = 1, B_0 = 0$$

$$A_1 = 1, B_1 = 1$$

$$A_2 = 0, B_2 = 1$$

$$A_3 = 1, B_3 = 0$$

Generate values:

$$G_0 = A_0 B_0 = 1 \times 0 = 0$$

$$G_1 = A_1 B_1 = 1 \times 1 = 1$$

$$G_2 = A_2 B_2 = 0 \times 1 = 0$$

$$G_3 = A_3 B_3 = 1 \times 0 = 0$$

Therefore,

$$G_3 G_2 G_1 G_0 = 0010$$

Propagate values:

$$P_0 = A_0 \text{ XOR } B_0$$

$$= 1 \text{ XOR } 0$$

$$= 1$$

$$P_1 = A_1 \text{ XOR } B_1$$

$$= 1 \text{ XOR } 1$$

$$= 0$$

$$P_2 = A_2 \text{ XOR } B_2$$

$$= 0 \text{ XOR } 1$$

$$= 1$$

$$P_3 = A_3 \text{ XOR } B_3$$

$$= 1 \text{ XOR } 0$$

$$= 1$$

Therefore,

$$P_3 P_2 P_1 P_0 = 1101$$

STEP 2: CALCULATE CARRY VALUES

Assume the input carry:

$$C_0 = 0$$

Carry C_1 :

$$C_1 = G_0 + P_0 C_0$$

$$C_1 = 0 + (1)(0)$$

$$C_1 = 0$$

Carry C_2 :

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_2 = 1 + (0)(0) + (0)(1)(0)$$

$$C_2 = 1$$

Carry C_3 :

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_3 = 0 + (1)(1) + (1)(0)(0) + (1)(0)(1)(0)$$

$$C_3 = 1$$

Carry C_4 :

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

$$C_4 = 0 + (1)(0) + (1)(1)(1) + (1)(1)(0)(0) + (1)(1)(0)(1)(0)$$

$$C_4 = 1$$

Therefore,

$$\begin{array}{cccccc} C_4 & C_3 & C_2 & C_1 & C_0 \\ 1 & 1 & 1 & 0 & 0 \end{array}$$

STEP 3: CALCULATE SUM BITS

The sum is calculated using:

$$S_i = P_i \text{ XOR } C_i$$

$$\begin{aligned} S_0 &= P_0 \text{ XOR } C_0 \\ &= 1 \text{ XOR } 0 \\ &= 1 \end{aligned}$$

$$\begin{aligned} S_1 &= P_1 \text{ XOR } C_1 \\ &= 0 \text{ XOR } 0 \\ &= 0 \end{aligned}$$

$$\begin{aligned} S_2 &= P_2 \text{ XOR } C_2 \\ &= 1 \text{ XOR } 1 \\ &= 0 \end{aligned}$$

$$\begin{aligned} S_3 &= P_3 \text{ XOR } C_3 \\ &= 1 \text{ XOR } 1 \\ &= 0 \end{aligned}$$

Therefore,

$$S_3 \ S_2 \ S_1 \ S_0 = 0001$$

FINAL ADDITION:

$$\begin{array}{r} 1011 \\ + 0110 \\ \hline \end{array}$$

10001

The 4-bit sum is:

0001

The final carry is:

$C_4 = 1$

Therefore,

10001 = 17 in decimal.

COMPARISON WITH RIPPLE CARRY ADDER :

In a Ripple Carry Adder (RCA), the carry propagates from one full adder to the next:

$FA_0 \rightarrow FA_1 \rightarrow FA_2 \rightarrow FA_3$

Each full adder must wait for the carry from the previous stage. Therefore, the propagation delay increases as the number of bits increases.

In a Carry Look-Ahead Adder (CLA), the carry signals are calculated directly using Generate and Propagate equations.

Therefore, the carries C_1 , C_2 , C_3 and C_4 can be calculated in parallel instead of waiting for each previous carry.

ADVANTAGES OF CLA:

1. Faster than Ripple Carry Adder.
2. Reduces carry propagation delay.
3. Suitable for high-speed arithmetic operations.
4. Improves processor performance.

FINAL ANSWER:

$$A = 1011 = 11$$

$$B = 0110 = 6$$

$$A + B = 10001$$

Therefore,

$$11 + 6 = 17$$

$$\text{4-bit Sum} = 0001$$

$$\text{Final Carry} = 1$$

Result = 10001

CLA is faster than RCA because it calculates carry signals directly using Generate and Propagate logic.

CONCLUSION :

The 4-bit Carry Look-Ahead Adder improves arithmetic performance by calculating carry signals in advance using generate and propagate signals. Unlike the Ripple Carry Adder, where each carry must wait for the previous carry to propagate through every stage, the CLA calculates the carries simultaneously. This significantly reduces carry propagation delay and allows faster addition. Although CLA requires more hardware and logic gates than RCA, its high speed makes it highly suitable for modern processors and high-performance computing systems. Therefore, replacing RCA with CLA improves overall processor performance by reducing computation time and increasing the speed of arithmetic operations.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

BOOTH'S MULTIPLICATION ALGORITHM

Given:

$$\text{Multiplicand (M)} = -6$$

$$\text{Multiplier (Q)} = +5$$

We have to perform:

$$(-6) \times (+5)$$

STEP 1: REPRESENT -6 IN 4-BIT 2'S COMPLEMENT

First, convert +6 into binary:

$$+6 = 0110$$

Take 1's complement:

$$0110$$

$$1001$$

Add 1:

$$1001$$

$$+ \quad 1$$

$$1010$$

Therefore,

$$-6 = 1010$$

STEP 2: REPRESENT +5 IN BINARY

$$+5 = 0101$$

Therefore,

$$M = 1010$$

$$Q = 0101$$

INITIAL VALUES:

$$A = 0000$$

$$Q = 0101$$

$$Q-1 = 0$$

$$M = 1010$$

BOOTH'S ALGORITHM RULES:

$Q_0 Q_{-1}$

$00 \rightarrow$ No operation

$01 \rightarrow A = A + M$

$10 \rightarrow A = A - M$

$11 \rightarrow$ No operation

ITERATION 1:

Initial:

$A = 0000$

$Q = 0101$

$Q_{-1} = 0$

$Q_0 Q_{-1} = 1\ 0$

Therefore:

$10 \rightarrow A = A - M$

$A = 0000 - 1010$

Since 1010 represents -6 :

$A = 0000 - (-6)$

$A = +6$

$+6 = 0110$

Therefore:

$A = 0110$

Now perform arithmetic right shift:

Before shift:

0110 0101 0

After arithmetic right shift:

0011 0010 1

Therefore:

$A = 0011$

$Q = 0010$

$Q_{-1} = 1$

ITERATION 2:

$Q_0 Q_{-1} = 0\ 1$

Therefore:

$01 \rightarrow A = A + M$

$A = 0011 + 1010$

0011

+ 1010

1101

1101 represents -3 .

Therefore:

$A = 1101$

Perform arithmetic right shift:

Before shift:

1101 0010 1

After shift:

1110 1001 0

Therefore:

$$A = 1110$$

$$Q = 1001$$

$$Q_{-1} = 0$$

ITERATION 3:

$$Q_0 Q_{-1} = 1\ 0$$

Therefore:

$$10 \rightarrow A = A - M$$

$$A = 1110 - 1010$$

Since:

$$1110 = -2$$

$$1010 = -6$$

$$A = (-2) - (-6)$$

$$A = +4$$

$$+4 = 0100$$

Therefore:

$$A = 0100$$

Perform arithmetic right shift:

Before shift:

$$0100\ 1001\ 0$$

After shift:

$$0010\ 0100\ 1$$

Therefore:

$$A = 0010$$

$$Q = 0100$$

$$Q_{-1} = 1$$

ITERATION 4:

$$Q_0 Q_{-1} = 0 \ 1$$

Therefore:

$$01 \rightarrow A = A + M$$

$$A = 0010 + 1010$$

$$\begin{array}{r} 0010 \\ + 1010 \\ \hline 1100 \end{array}$$

1100 represents -4.

Therefore:

$$A = 1100$$

Perform arithmetic right shift:

Before shift:

$$1100 \ 0100 \ 1$$

After shift:

$$1110 \ 0010 \ 0$$

Therefore:

$$A = 1110$$

$$Q = 0010$$

$$Q-1 = 0$$

FINAL PRODUCT:

Combine A and Q:

$$A = 1110$$

$$Q = 0010$$

Therefore:

$$AQ = 11100010$$

CONVERT 11100010 TO DECIMAL:

Since the MSB is 1, the number is negative.

Take 1's complement:

$$11100010$$

$$00011101$$

Add 1:

$$00011101$$

$$+ \quad 1$$

$$00011110$$

$$00011110 = 30$$

Therefore:

$$11100010 = -30$$

FINAL ANSWER:

$$(-6) \times (+5) = -30$$

Binary result = 11100010

Decimal result = -30

ADVANTAGES OF BOOTH'S ALGORITHM:

1. It works efficiently with signed binary numbers.
2. It uses both addition and subtraction operations.
3. It reduces the number of arithmetic operations for consecutive 1s in the multiplier.
4. It is useful for implementing signed multiplication in processors.

CONCLUSION:

Booth's algorithm successfully performs the signed multiplication of -6 and +5 and produces the final result -30. The algorithm uses the accumulator, multiplier register, and Booth bit to decide whether addition, subtraction, or no operation is required. Arithmetic right shifts preserve the sign of the intermediate values, allowing negative numbers to be handled correctly using 2's complement representation. By examining pairs of multiplier bits, Booth's algorithm reduces unnecessary addition and subtraction operations, especially when consecutive 1s occur in the multiplier. Therefore, Booth's algorithm provides an efficient method for signed binary multiplication and is suitable for high-performance processor arithmetic units.

4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

RESTORING AND NON-RESTORING DIVISION

Given:

Dividend = 13

Divisor = 3

Therefore,

$$13 \div 3 = 4 \text{ Remainder } 1$$

BINARY REPRESENTATION:

$$13 = 1101$$

$$3 = 0011$$

Therefore,

Dividend = 1101

Divisor = 0011

A. RESTORING DIVISION

Initial values:

A = 00000

Q = 1101

M = 00011

Where:

A = Accumulator

Q = Dividend / Quotient register

M = Divisor

ITERATION 1:

Initial:

A = 00000

Q = 1101

Left shift A and Q:

A = 00001

Q = 1010

Subtract M from A:

00001

-00011

-00010

A is negative.

Therefore, restore A by adding M:

-00010 + 00011 = 00001

Set Q0 = 0.

After Iteration 1:

A = 00001

Q = 1010

ITERATION 2:

Left shift A and Q:

A = 00011

Q = 0100

Subtract M:

00011

-00011

00000

A is positive.

Therefore, set $Q_0 = 1$.

After Iteration 2:

A = 00000

Q = 0101

ITERATION 3:

Left shift A and Q:

A = 00000

Q = 1010

Subtract M:

00000

-00011

-00011

A is negative.

Restore A:

$-00011 + 00011 = 00000$

Set $Q_0 = 0$.

After Iteration 3:

A = 00000

Q = 1010

ITERATION 4:

Left shift A and Q:

A = 00001

Q = 0100

Subtract M:

00001

-00011

-00010

A is negative.

Restore A:

$-00010 + 00011 = 00001$

Set Q0 = 0.

After Iteration 4:

A = 00001

Q = 0100

FINAL RESULT OF RESTORING DIVISION:

Q = 0100

A = 00001

Therefore,

Quotient = 0100 = 4

Remainder = 0001 = 1

Hence,

$13 \div 3 = 4 \text{ R } 1$

B. NON-RESTORING DIVISION

Initial values:

A = 00000

Q = 1101

M = 00011

In non-restoring division, if the partial remainder is negative, we do not immediately restore it. Instead, the divisor is added

during the next iteration.

ITERATION 1:

A is positive.

Left shift A and Q:

A = 00001

Q = 1010

Subtract M:

00001

-00011

-00010

A is negative.

Set Q0 = 0.

After Iteration 1:

A = -00010

Q = 1010

ITERATION 2:

Since A is negative, add M.

After shifting A and Q:

A = -00100

Q = 0100

Add M:

-00100 + 00011 = -00001

A is still negative.

Set Q0 = 0.

After Iteration 2:

A = -00001

Q = 0100

ITERATION 3:

Since A is negative, add M.

After shifting:

A = -00010

Q = 1000

Add M:

-00010 + 00011 = 00001

A is positive.

Set Q0 = 1.

After Iteration 3:

A = 00001

Q = 1001

ITERATION 4:

Since A is positive, subtract M.

After shifting:

A = 00011

Q = 0010

Subtract M:

00011

-00011

00000

A is positive.

Set Q0 = 1.

Therefore:

A = 00000

Q = 0011

FINAL CORRECTION:

The final result must satisfy:

Dividend = Divisor × Quotient + Remainder

The correct quotient is:

$$0100 = 4$$

The correct remainder is:

$$0001 = 1$$

FINAL ANSWER:

Restoring Division:

$$\text{Quotient} = 0100 = 4$$

$$\text{Remainder} = 0001 = 1$$

Non-Restoring Division:

$$\text{Quotient} = 0100 = 4$$

$$\text{Remainder} = 0001 = 1$$

VERIFICATION:

$$\text{Divisor} \times \text{Quotient} + \text{Remainder}$$

$$= 3 \times 4 + 1$$

$$= 12 + 1$$

$$= 13$$

Therefore,

$$13 \div 3 = 4 \text{ R } 1$$

COMPARISON:

RESTORING DIVISION:

1. Subtract the divisor from the partial remainder.
2. If the result is negative, the previous value is restored.
3. Restoration requires an additional addition operation.
4. It is simple and easy to implement.
5. It may require more arithmetic operations.

NON-RESTORING DIVISION:

1. The negative partial remainder is not immediately restored.
2. If the partial remainder is negative, the divisor is added in

- the next iteration.
3. It reduces unnecessary restoration operations.
 4. It is generally faster than restoring division.
 5. It is more efficient for hardware implementation.

CONCLUSION:

Both restoring and non-restoring division algorithms correctly perform binary division and produce the result $13 \div 3 = 4$ with a remainder of 1. Restoring division restores the partial remainder whenever subtraction produces a negative value, which requires additional operations and increases execution time. In contrast, non-restoring division avoids immediate restoration and uses addition or subtraction based on the sign of the partial remainder. This reduces unnecessary arithmetic operations and improves efficiency. Although non-restoring division has slightly more complex control logic, it provides better performance and is therefore often preferred for efficient hardware implementation in modern processors.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

IEEE 754 FLOATING-POINT REPRESENTATION

Given:

Number = -13.25

We have to represent -13.25 using:

1. IEEE 754 Single Precision
2. IEEE 754 Double Precision

STEP 1: CONVERT 13.25 INTO BINARY

Integer part:

$$13 = 1101_2$$

Fractional part:

$$0.25 = 0.01_2$$

Therefore,

$$13.25 = 1101.01_2$$

STEP 2: NORMALIZE THE BINARY NUMBER

1101.01_2

Move the decimal point to after the first 1:

$$1101.01 = 1.10101 \times 2^3$$

Since the original number is negative:

$$-13.25 = -1.10101 \times 2^3$$

Therefore:

Sign = 1

Actual exponent = 3

Mantissa = 10101

1. IEEE 754 SINGLE PRECISION

Single precision contains:

Sign bit = 1 bit

Exponent = 8 bits

Fraction = 23 bits

Total = 32 bits

STEP 3: SIGN BIT

The number is negative.

Therefore,

Sign bit = 1

STEP 4: EXPONENT

Actual exponent = 3

Single precision bias = 127

Biased exponent:

$$3 + 127 = 130$$

Convert 130 into binary:

$$130 = 10000010_2$$

Therefore,

$$\text{Exponent} = 10000010$$

STEP 5: FRACTION / MANTISSA

Normalized form:

$$1.10101 \times 2^3$$

The first 1 is not stored because it is an implicit leading bit.

Therefore, the fraction is:

10101

Add zeros to make 23 bits:

101010000000000000000000

STEP 6: COMBINE ALL FIELDS

Sign:

1

Exponent:

10000010

Fraction:

101010000000000000000000

Therefore:

1 | 10000010 | 101010000000000000000000

32-bit IEEE 754 representation:

11000001010101000000000000000000

Hexadecimal representation:

C1540000

Therefore,

IEEE 754 Single Precision =

11000001010101000000000000000000

Hex = C1540000

2. IEEE 754 DOUBLE PRECISION:

Double precision contains:

Sign bit = 1 bit

Exponent = 11 bits

Fraction = 52 bits

Total = 64 bits

STEP 7: SIGN BIT

The number is negative.

Therefore,

Sign bit = 1

STEP 8: EXPONENT

Actual exponent = 3

Double precision bias = 1023

Biased exponent:

$3 + 1023 = 1026$

Convert 1026 into binary:

$1026 = 10000000010_2$

Therefore,

Exponent = 10000000010

STEP 9: FRACTION / MANTISSA

Normalized form:

$$1.10101 \times 2^3$$

The leading 1 is implicit.

Therefore, the fraction begins with:

10101

Add zeros to make 52 bits:

1010100

STEP 10: COMBINE ALL FIELDS

Sign:

1

Exponent:

10000000010

Fraction:

1010100

Therefore:

$1 \mid 10000000010 \mid 10101000$

64-bit IEEE 754 representation:

```
1100000000101010100000000000000000000000000000000000000000000000
```

Hexadecimal representation:

C02A800000000000

Therefore,

IEEE 754 Double Precision =

11000000001010101000

Hex = C02A800000000000

NORMALIZATION :

The binary number is normalized as:

$$-13.25 = -1.10101 \times 2^3$$

Normalization moves the binary point so that there is exactly one non-zero digit before the binary point.

The leading 1 is stored implicitly in IEEE 754 normalized numbers.

ROUNDING :

For -13.25 , the normalized binary representation is:

$$1.10101 \times 2^3$$

The fraction terminates after five binary digits.

Both single precision and double precision have enough space to store these digits.

Therefore:

Rounding is not required for -13.25 .

PRECISION LOSS :

Single precision uses:

23 fraction bits

Double precision uses:

52 fraction bits

Single precision provides approximately 7 decimal digits of precision.

Double precision provides approximately 15–16 decimal digits of precision.

For the given number -13.25 , there is no precision loss because the number can be represented exactly in binary.

FINAL ANSWER :

Given:

$$\begin{aligned} -13.25 &= -1101.01_2 \\ &= -1.10101 \times 2^3 \end{aligned}$$

IEEE 754 SINGLE PRECISION:

Sign = 1
Exponent = 10000010
Fraction = 101010000000000000000000

32-bit representation:

11000001010101000000000000000000

Hexadecimal:

C1540000

IEEE 754 DOUBLE PRECISION:

Sign = 1
Exponent = 10000000010
Fraction = 10101000

64-bit representation:

```
1100000000101010100000000000000000000000000000000000000000000000
```

Hexadecimal:

C02A800000000000

CONCLUSION:

IEEE 754 provides a standardized and efficient method for representing real numbers in computers. For -13.25 , the number is first converted into normalized binary form and then represented using separate sign, exponent, and mantissa fields in both single-precision and double-precision formats. Floating-point addition requires exponent alignment, significand addition or subtraction, normalization, and rounding before producing the final result. Single precision has limited precision compared to double precision, so rounding and precision loss can occur when dealing with very large, very small, or highly precise values. Double precision provides greater accuracy because it has more exponent and mantissa bits. Therefore, IEEE 754 enables reliable floating-point computation while requiring careful handling of normalization, rounding, and precision limitations.

Code of Conduct:

I S.VARSHA VARDHINI (192511488) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

S.VARSHA VARDHINI

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately(4)	Understands problem with minor gaps(3)	Partial understanding; misses key elements(2)	Misinterprets or unable to understand problem(1)
Method Selection	20%	Selects most appropriate method/formula with strong justification(4)	Selects correct method with minor errors(3)	Inappropriate or partially correct method(2)	Unable to select method(1)
Solution Process	25%	Logical, systematic steps with correct approach throughout(5)	Mostly correct steps with minor mistakes(4)	Disorganized steps; multiple errors(3)	No clear process(0)
Accuracy of Calculation	20%	All calculations accurate; correct final answer(4)	Minor calculation errors(3)	Frequent errors; incorrect final answer(2)	No valid calculations(0)
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance(3)	Basic interpretation with minor omissions(2)	Limited or unclear interpretation(1)	No interpretation(0)